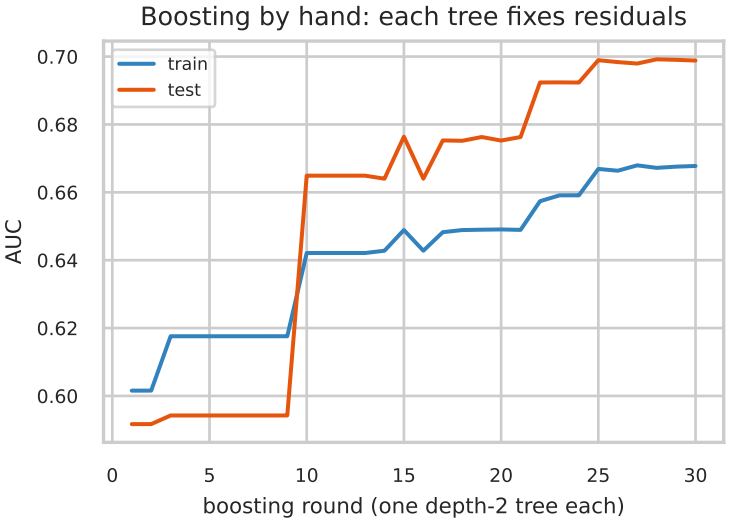
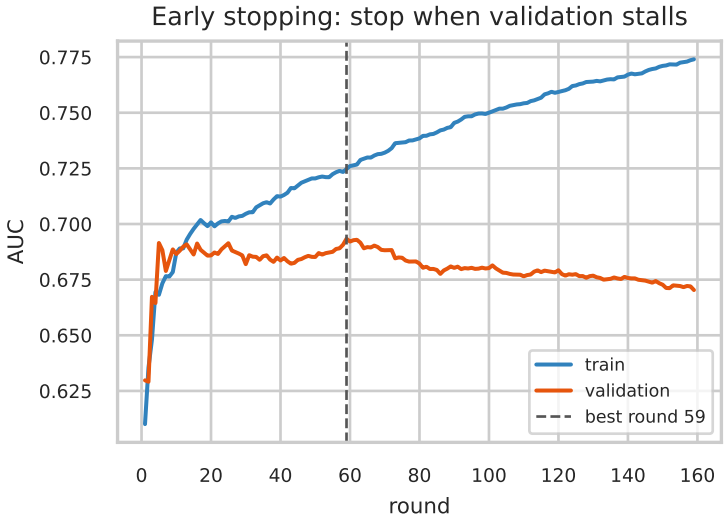
Day 17 Cheat Sheet: Gradient Boosting with XGBoost

Week 3: boosting, learning rate, early stopping, and CV tuning on 30-day readmissions

TERM	WHAT IT MEANS	CODE
Boosting	add trees one at a time; each fixes current errors	XGBClassifier()
Residual / gradient	actual minus predicted risk: what is still wrong	y - p
learning_rate (eta)	how big a step each tree takes; small = needs more	learning_rate=0.03
n_estimators	max rounds; with early stopping, set high	n_estimators=2000
early_stopping_rounds	stop when validation AUC has not improved for N rounds	early_stopping_rounds=100
max_depth	depth of EACH tree; 1 = additive, deeper = interactions	max_depth=2
subsample / colsample	row / column share per tree (adds randomness)	subsample=0.8
min_child_weight	blocks tiny leaves (like min_samples_leaf)	min_child_weight=5
eval_set	data XGBoost scores every round (train + validation)	eval_set=[(X_val, y_val)]
scale_pos_weight	upweights the rare class (hurts calibration)	neg / pos





THIS DATA (same simulated days 14-16 data, seed 14): 4,000 discharges, 9.0% readmitted, 3,000 train / 1,000 test
By hand: 30 depth-2 trees, step 0.3, test AUC 0.59 after 1 tree, 0.70 after 30
XGBoost lr 0.05, depth 3, early stopping: best round 59 of 2000, validation AUC 0.693, test AUC 0.700
Greedy (depth 8, lr 0.3, no stopping): train AUC 1.00, test AUC 0.571. Pure memorization
CV pick: {'learning_rate': 0.01, 'max_depth': 1}, CV AUC 0.638, test AUC 0.709
Scoreboard: logistic 0.723 | forest 0.716 | XGBoost 0.709 | tree 0.649

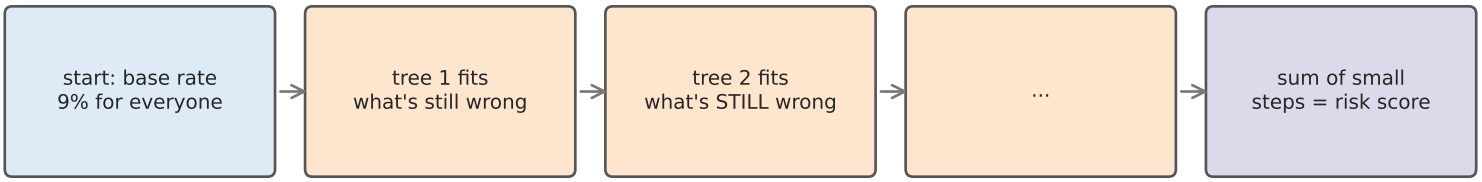
HABITS THAT KEEP YOU HONEST
- Early stopping uses a validation split carved from TRAINING data, never the test set
- Lower learning rate + more rounds is safer than a high rate; let early stopping pick the count
- CV chose depth 1 (no interactions): the simulated risk is additive, so logistic is hard to beat
- Boosting wins on real EHR data with interactions and thresholds; always keep a simple baseline
- Report test AUC once, at the end, for the model you picked with CV

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=2000, learning_rate=0.05, max_depth=3, subsample=0.8,
                    eval_metric="auc", early_stopping_rounds=100, random_state=14)
xgb.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
print(xgb.best_iteration, xgb.best_score)
```

Infographic: Trees in a Relay

Simulated readmissions data from days 14-16. Same split, a boosted model

Gradient boosting = a relay of small trees



Forest: many independent trees, averaged. Boosting: trees in a SEQUENCE, each one shrunk by the learning rate.

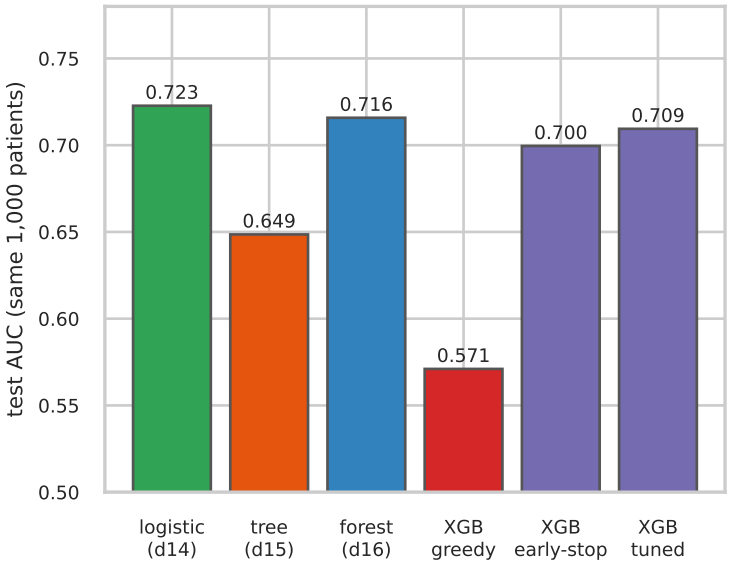
GREEDY XGB
train 1.00
test 0.57

EARLY STOP
round 59
test AUC 0.70

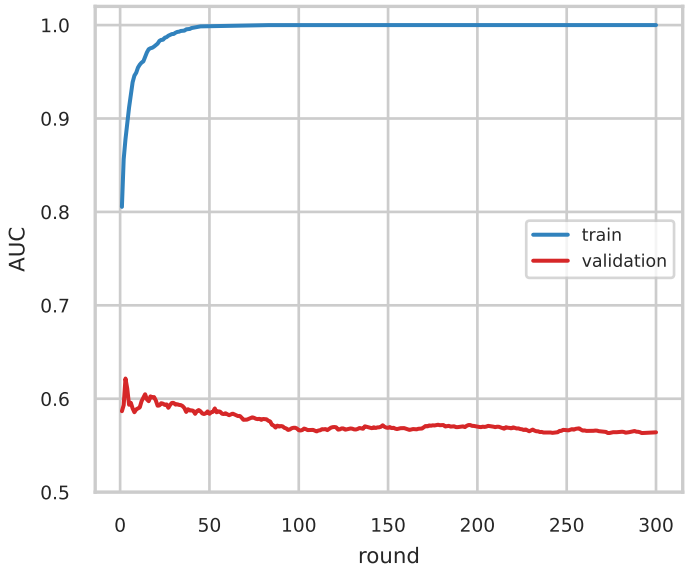
CV PICKS
depth 1, lr 0.01
test AUC 0.71

VS LOGISTIC
0.709 vs 0.723
still a tie

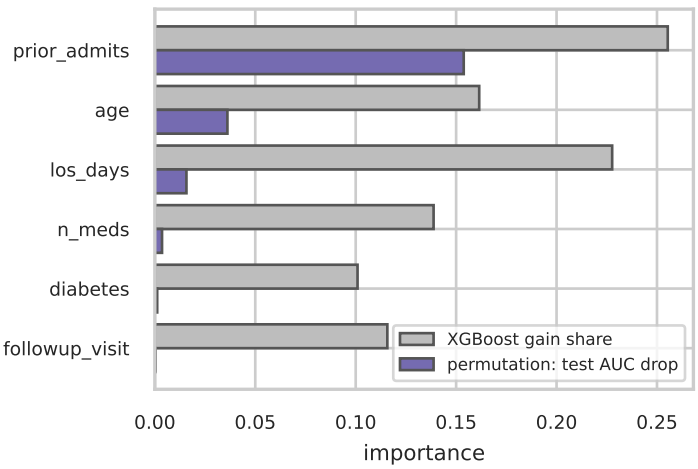
Scoreboard: boosting ties, does not win



Depth 8, lr 0.3: memorizes (train 1.00)



Gain vs permutation importance



Takeaways
- Boosting builds trees in sequence, each one correcting the last; a forest averages independent trees
- Unchecked, it memorizes (train 1.00, test 0.57)
- Early stopping + small steps fix that
- prior_admits leads every importance measure
- Tomorrow: imbalanced classes, thresholds and precision-recall

Hands-On Worksheet: ds-genai-90-days / day17

60-90 minutes | notebook: day17/xgboost.ipynb + day17/README.md

[ ] mkdir day17, conda activate ds90, pip install xgboost

[ ] Rebuild the days 14-16 data and the same split (snippet 1)

[ ] Boost by hand: 30 depth-2 trees fit to residuals (snippet 2)

[ ] Plot train vs test AUC by round; explain the learning rate

[ ] Split a validation set from TRAIN; fit XGBoost with early stopping (snippet 3)

[ ] Record best_iteration and validation AUC; plot evals_result()

[ ] Fit the greedy model (depth 8, lr 0.3) and plot its curves (snippet 4)

[ ] GridSearchCV: max_depth x learning_rate, 5-fold, AUC

[ ] Heatmap of CV AUC; write why depth 1 wins on this data

[ ] Six-model scoreboard on the same test set (snippet 5)

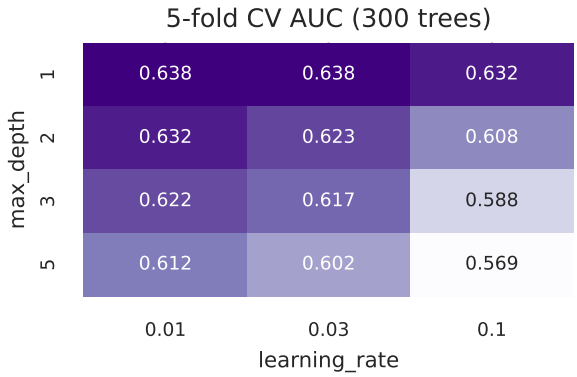
[ ] Gain vs permutation importance side-by-side bar chart

[ ] README: 3 charts, 4 findings, honest 'boosting tied logistic' note

[ ] git add day17 && git commit && git push (snippet 6)

[ ] Stretch: load XGBoost scores into Snowflake, compare top-decile lift (snippet 7)

```
xgb = XGBClassifier(n_estimators=2000,
    learning_rate=0.05, max_depth=3,
    eval_metric="auc",
    early_stopping_rounds=100)
xgb.fit(X_tr, y_tr,
    eval_set=[(X_val, y_val)])
```



Best round: _____ Validation AUC: _____ Greedy train / test AUC: _____ / _____

Best CV params: _____ CV AUC: _____ XGB / forest / logistic test AUC: ____ / ____ / ____

Day 17 Self-Check Quiz

1. In one sentence each: how does a forest combine trees, and how does boosting?

   _____

2. What does each new tree in gradient boosting try to predict?

   _____

3. What does learning_rate do, and what happens if you lower it?

   _____

4. Why must early stopping use a validation set and not the test set?

   _____

5. The greedy model has train AUC 1.00 and test AUC 0.57. Name two fixes.

   _____

6. CV picked max_depth 1. What does a depth-1 boosted model assume?

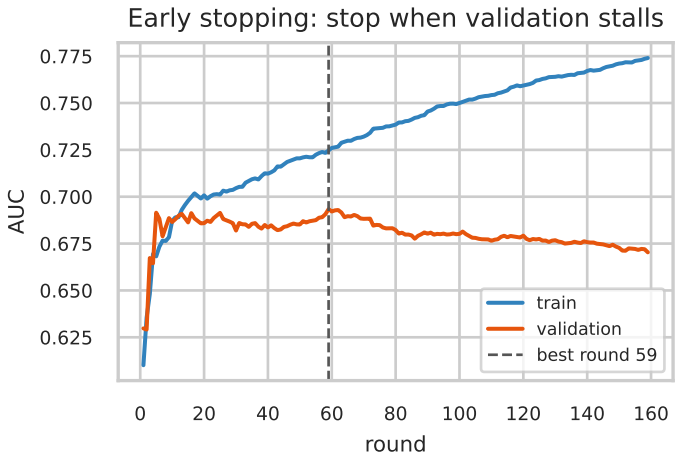
   _____

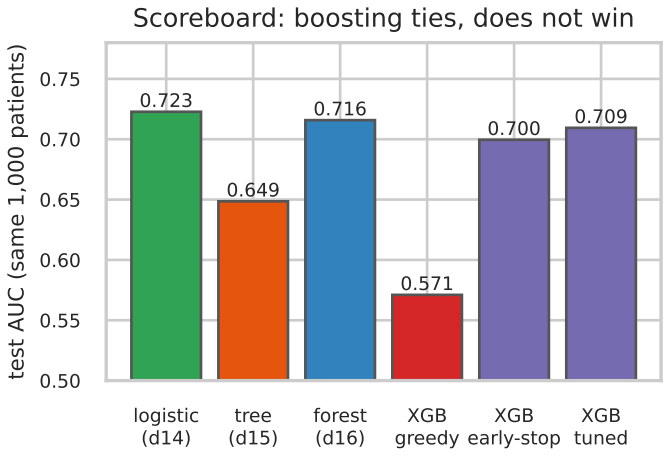
7. XGBoost tied logistic regression here. Was the exercise a failure?

   _____

8. Gain says los_days is #2; permutation says age is #2. Why can they differ?

   _____





Answers:
1 forest: many independent trees on bootstrap samples, averaged; boosting: trees added one after another, each correcting the current errors
2 the residuals (gradient of the loss): the gap between the actual outcome and the model's current predicted risk
3 it shrinks each tree's contribution; lower = smoother, needs more rounds, usually generalizes better (pair with early stopping)
4 if the test set chooses the stopping round, it is part of training and its AUC is optimistic; keep test untouched until the end
5 any two: shallower trees, smaller learning rate, early stopping, min_child_weight / subsample, fewer rounds
6 no interactions: each tree splits on one feature, so risk is a sum of per-feature effects, the same shape as logistic regression
7 no: the simulated truth is additive, so a linear model is ideal; the lesson is to keep a simple baseline and let CV decide
8 gain counts loss reduction during training (favors many-valued columns like los_days); permutation measures lost test AUC directly